Info 1 - 2015 `// Viel Spass damit und lernt brav ;)`

1. 1. Vermischtes zur Einführung

Markieren Sie *alle* zutreffenden Aussagen:

- Top-Down-Entwurf besteht darin, Details erst grob, dann feingliedrig zu sortieren.
- Dem Inhalt einer Speicherzelle kann man nicht ansehen, ob sie eine Zahl oder einen Befehl speichert..
- Beim euklidischen Algorithmus wird dieser Block iteriert: `{ x = y; y = r; r = x % y; }`
- Beim Testen wird das Programm auf allen erlaubten Eingaben ausgeführt.
- Beim Pipelining werden Eingaben eines Prozesses zur Ausgabe eines anderen geleitet.
- Beim Laden wird die Befehlsfolge in aufeinanderfolgende Register kopiert.
- Editieren und Testen gehören zu jedem Programmentwicklungszyklus.
- `public static void main(String[] arks)` ist Kopf der `main`-Methode.

2. 1. Zweierpotenzen

Ergänzen Sie untenstehende `main`-Methode so, dass der Aufruf `java Zweierpotenzen` n die Liste der Zweierpotenzen von $2^0 = 1$ bis 2^n zeilenweise ausgibt.

```
public class Zweierpotenzen {
    public static void main (String[] args) {
        // ---------------------------
        .....    // soviele Codezeilen wie nötig
        .....    // (angemessen EINGERÜCKT)
        ..
        // ---------------------------
   }
}
```

Beispiel `java Zweierpotenzen 3` gibt diese Liste aus:

```
1
2
4
8
```

Algorithmische Idee (bitte GENAU diese Variablennamen benutzen):

initialisiere:
- Kommandozeilenargument (interpretiert als `int`-Wert) in `n` speichern
- `int`-Variable `p` als Akkumulator initialisieren

iteriere so oft wie nötig:
- `p` ausgeben und geeignet aktualisieren

finalisiere: entfällt

3. 1. Quadratzahlentabelle

Die `int`-Variable `n` speichere die ganze Zahl n.

Die ersten n positiven Zahlen sollen zusammen mit ihren Quadraten jeweils auf einer Zeile

ausgegeben werden.

Beispiel für $n = 4$:

```
1 : 1
2 : 4
3 : 9
4 : 16
```

Im Fall $n \leq 0$ soll nichts ausgegeben werden.

"Programmieren" Sie dies durch Auswahl der richtigen Codezeilen, und zwar einmal mit einer `for`-Schleife und einmal mit einer `do-while`-Schleife.

Achten Sie auf den Unterschied zwischen `System.out.print(..)` und `System.out.println(..)`.

- mit einer for-Schleife:

```
for (int i = 1; i < n+1; i++) {
System.out.print(i + ": ");
System.out.println( i^2);
}
```

- mit einer do-while-Schleife:

```
int i = 1;
do {
System.out.print(i + ": ");
System.out.println( i^2);
i++;
} while ( i < n+1);
```

4. 2. Vermischtes zu elementaren Daten

Markieren Sie *alle* zutreffenden Aussagen:

- Der ASCII-Standard legt die Anordnung der 256 Zeichen fest.
- Zu jedem Datentyp gibt es ein Informationssystem.
- Java-Code darf beliebige Unicode-Zeichen enthalten.
- `int`-Werte können verlustfrei in `float`-Variablen gespeichert werden.
- `5/4 == 1` hat den Wert `true`.
- `Integer.MAX_VALUE++` verursacht keinen Ganzzahlüberlauf.
- Anstatt `if (b > 0 && a/b < 2) ..` schreibt man besser `if (a/b < 2 && b > 0) ...`
- Ein Gleitkomma-Überlauf ist kein Laufzeitfehler.

5. 2. Integer-Arithmetik

Sei `d` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
1:  System.out.println(-d);
2:  System.out.println(d-1);          // dropdown menue mit loesungen
3:  System.out.println(1+2*d);
4:  System.out.println(-(2*d));
```

6. 2. Verarbeitung elementarer Daten (I1-ID: 93rdw3918rg0)

In einer Klasse sind einige Methoden definiert:

```
public static void a() {
    char a = 'a';
    a++;
    int b = (int) a;
    b++;
    System.out.print( (char) ('A' + b - a));
}
public static boolean getTrue() {
    System.out.print("T");
    return true;
}
public static boolean getFalse() {
    System.out.print("F");
    return true;
}
public static void q(int i) {
    int j = 43;
    System.out.print("G");
    if ( i / j > 0) System.out.print("Q");
    else System.out.print("R");
}
public static void d() {
    double dd = Double.MIN_VALUE;
    dd *= 2;
    dd -= Double.MIN_VALUE;
    dd /= 2;
    System.out.print(dd);
}
```

Welche Ausgaben bewirken diese Aufrufe (in der `main`-Methode der gleichen Klasse)?

```
a();



boolean val = getTrue() || getFalse();



val = getTrue() ^ getTrue();



System.out.print( val);



q(42);



d();
```

7. 3. Rekursion

Betrachten Sie folgende Klasse:

```
 1:  class Myst {
 2:      static void ery(int n) {
 3:          if (n == 0)
 4:              return;
 5:          ery( n-1);
 6:          A();
 7:          yre( n-1);
 8:      }
 9:      static void yre(int n) {
10:          if (n == 0)
11:              return;
12:          ery( n-1);
13:          B();
```

```
14:          yre( n-1);
15:      }
16:      static void A() {
17:          System.out.print("A");
18:      }
19:      static void B() {
20:          System.out.print("B");
21:      }
22:      public static void main(String[] args) {
23:          int n = Integer.parseInt(args[0]);
24:          ery(n);
25:          System.out.println();
26:      }
27:  }
```

1. Geben Sie die ersten 8 Zeichen der Ausgabe zum Aufruf `java Myst 4` an:

2. Aus wievielen Zeichen besteht der Ausgabestring beim Aufruf `java Myst 7` (*ohne* den abschließenden Zeilenwechsel durch Zeile 25)?

3. Wird Zeile 14 durch `yre( n-2);` ersetzt, so führt beispielsweise der Aufruf `java Myst 4` zu einem Fehler.
   Welche *andere* Zeile müsste korrigiert werden, um den Fehler zu beseitigen? Geben Sie nur die Nummer an:

4. Werde Zeile 12 durch `ery( n-2);` und zusätzlich Zeile 14 durch `yre( n-2)` ersetzt.
   Geben Sie das maximale Argument `n` an, bei dem der Aufruf `java Myst n` *nicht* zu einem Fehler führt:

8. 3. Grafik-Bibliothek benutzen

Die Klasse `public class StdDraw` bietet Methoden zum Zeichnen geometrischer Objekte:

- die quadratische Zeichenfläche (Seitenlänge 1) wird in einem Fenster mit dem Titel "File" angezeigt
- Koordinaten wie üblich (z.B. gehen die blauen Strecken oben alle durch den Punkt $(x, y) = (1, 0)$, die diagonal gegenüberliegende Quadratecke ist $(0, 1)$)
- gezeichnet wird mit der zuletzt gesetzen Stiftfarbe, wobei später Gezeichnetes Früheres (teilweise) überdecken kann

`void setPenColor(Color c)`	Stiftfarbe setzen
`BLACK, RED, BLUE`	in `StdDraw` definierte `Color`-Konstanten (schwarz, rot, blau)
`void line(double x0, double y0, double x1, double y1)`	Strecke zwischen Punkten `(x0,y0)` und `(x1,y1)`
`void square(double x, double y, double r)`	Quadrat: Mittelpunkt `(x,y)` u. Seitenlänge `2*r` (parallel zu Koordinatenachsen)
`void circle(double x, double y, double r)`	Kreis mit Mittelpunkt `(x,y)` und Radius `r`

Nachfolgend der Ausschnitt einer Methode einer Clientklasse von `StdDraw`.
Ergänzen Sie Zeilen so, dass der Aufruf `zeichneB(n)` (z.B. in der `main`-Methode dieser Klasse) eine Figur erzeugt aus `n` mal `n` Kreisen usw.
Genau so wie oben gezeigt. Achten Sie dabei auf gut verständlichen Code.

```
static void zeichneB(int n) {
```

```
                                    // Ekelhafte Dropdown menues

    for (int i = 0; i < n; i++)                    StdDraw.circle(r*i,r*j,r/2);
      for (int j = 0; j < n; j++)                StdDraw.setPenColor(StdDraw.RED);
       StdDraw.setPenRadius(0.02);                  StdDraw.line(1.0,0.0,0,r+2*r*i);

         for (int i = 0; i < n; i++)              StdDraw.setPenColor(StdDraw.BLUE);
         double r = 1.0/(2*n);                      for (int j = 0; j < n; j++)
                                                     StdDraw.square(0.5,0.5,1);
}
```

9. 4. Felder: Fehler oder …?

Erzeugt untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich:
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich:
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich:

```
 1:  class FehlerOderWie {
 2:      public static void main(String[] args) {
 3:          int[] a = {2,3,5,7,11};
 4:          int[] b = new int[2];
 5:          b = a;
 6:          b[4] = -4;
 7:          if (a[4] >= 1) System.out.println(1);
 8:          else System.out.println(0);
 9:      }
10:  }
```

10. 4. Vermischtes zu Klassentypen

1. Objekte verstehen:

   ```
   public class Spieler {
       private int score;
       public Spieler( int s) {
           this.score = s;
       }
   }
   ```

   Zwei `Spieler`-Objekte werden instanziiert:

   ```
   Spieler a = new Spieler(50), b = new Spieler(50);
   ```

   Welchen Wert hat nun der Ausdruck a == b?

2. Fremden Code deuten:

   ```
   1:  public class FirstBpp extends hepno.users.Frame {
   2:      private hepno.users.Wring wrags;
   3:      public FirstBpp() {
   4:          wrags = new hepno.users.Wring();
   5:      }
   6:      public static void main(String[] args) {
   7:          FirstBpp app = new FirstBpp();
   8:      }
   9:  }
   ```

   Was bewirkt Zeile 2?:
   hepno.users.Wring wird als private-Objekt deklariert

3. Statische Methoden und Instanzmethoden:

```
public class Klasse {
    private final int zahl = 10;
    double a() {
        System.out.println(zahl);
        return c();
    }
    void b() {
        System.out.println(this);
    }
    double c() {
        double r = Math.random();
        System.out.println(r);
        return r;
    }
    void d() {
        a();
        a();
    }
    int e() {
        return zahl;
    }
}
```

Welche der obigen Methoden ist ihrer Natur nach eher eine statische Methode (und sollte daher mit `static` versehen werden)?
c

4. Fehlersuche:

```
public class Test {
    private int testCount;
    public static int getCount(){
        return testCount;
    }
    public Test(){
        testCount++;
    }
}
```

Obiger Code verursacht einen Compilezeitfehler. Was ist das Problem?
`getCount() ist statisch`

5. Tracing:
[Hinweis: Untenstehender Code ist syntaktisch korrekt, denn auf Klassenvariablen kann auch mit Hilfe von Objektreferenzen zugegriffen werden.]

```
public class Tset {
    static int tsetCount = 0;
    public Tset(){
        tsetCount++;
    }
    public static void main(String [] Args){
        // Hinweis: auf Klassenvariablen kann man auch
        tset t1 = new tset();
        System.out.print(t1.tsetCount + ", ");
        tset t2 = new tset();
        System.out.print(t1.tsetCount + " "+ t2.tsetCount + ", ");
        tset t3 = new tset();
        System.out.print(t1.tsetCount+ " "+ t2.tsetCount+ " "+ t3.tsetCount);
        System.out.println();
    }
}
```

Welche Ausgabe produziert der Code?

6. Begriffsverständnis: Wählen Sie aus dem Menü die einzige Aussage aus, die NICHT korrekt ist.

In einer Klasse darf es Methoden geben, die den gleichen Namen und gleiche Argumenttyplisten haben aber unterschiedliche Ergebnistypen.

11. 4. Iteration in einem Feld

Tragen Sie die Werte der Feldkomponenten nach Ausführung der `for`-Anweisung ein.

```
int[] d = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

for (int i = 0; i < 10; i++)
    d[i] = d[9-d[9-d[9-i]]];
```

vorher	0	1	2	3	4	5	6	7	8	9
nachher	9	8	7	6	5	5	6	7	8	9

12. 6. Stapelarbeit

Auf einem Stack werden *in gemischter Reihenfolge* 10 Push- und 10 Pop-Operationen ausgeführt.

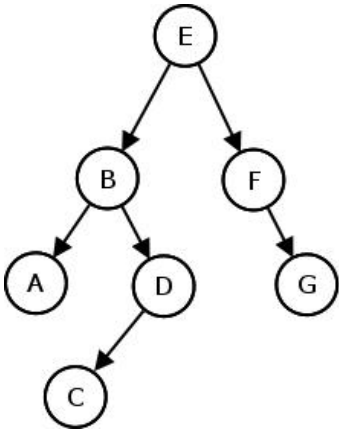
Es ist bekannt, dass die Push-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird.

Markieren Sie alle Ausgabelisten, die dabei entstehen können (das können 0, 1 oder mehrere sein):

1 4 7 9 8 6 5 3 0 2
4 3 2 1 0 9 8 7 6 5
2 5 6 7 4 8 9 3 1 0
4 6 8 7 5 3 2 9 0 1

13. 6. Binärer Suchbaum

Es geht um binäre Suchbäume, deren Schlüssel Zeichen in alphabetischer Anordnung sind.



1. Werden die Schlüssel obigen binären Suchbaums in Inorder-, Preorder- oder Postorder-Reihenfolge ausgegeben, so erscheint ein Schlüssel immer an gleicher Position.
   Welcher ist das?

2. Wählen Sie für die Schlüssel eine Einfügereihenfolge aus, bei der ein Binärbaum der Tiefe 2 entsteht:

14. 6. Vermischtes zu Algorithmen und Datenstrukturen

Markieren Sie *alle* zutreffenden Aussagen:

Binäre Suche erfordert $O(n)$ Vergleiche auf *unsortierten* Feldern.

Mit einem Stack kann man z.B. feststellen, ob ein Ausdruck korrekt geklammert ist.

Die abstrakte Datenstruktur Queue lässt sich nicht mit einfach verketteten Listen realisieren.

Quicksort ist auf vorsortierten Feldern besonders schnell.

Mergesort benötigt auf allen Feldern der Länge 2^m immer die gleiche Anzahl an Vergleichen.

Selectionsort benötigt auf allen Feldern der Länge 2^m immer die gleiche Anzahl an Vergleichen.

Binäre Bäume der Tiefe n haben mindestens $n-1$ und höchstens 2^n-1 *innere Knoten*.

Felder sind nicht als Grundlage der abstrakten Datenstruktur Stack geeignet.

15. 5. Verständnisfragen zu OOP

Sei p eine Referenzvariable vom Typ A. Was kann alles der dynamische Typ von p sein?

Markieren Sie *alle* Möglichkeiten, die Java erlaubt:

eine Erweiterungsklasse einer Klasse, die A implementiert (falls A ein Interface ist)
ein Interface, dass A als abstrakte Klasse erweitert (falls A eine Klasse ist)
eine Klasse, von der A abgeleitet ist (falls A eine Klasse ist)
eine Erweiterungsklasse von A (falls A eine Klasse ist)
ein elementarer Datentyp (falls A die zugehörige Wrapper-Klasse ist)
eine Klasse, die A implementiert (falls A ein Interface ist)

16. 5. OOP-Entwurf

Die Klasse `Ente` erbt von der Klasse `Vogel`. In `Vogel` gibt es eine Instanzmethode `fliege(..)`, in `Ente` gibt es die Instanzmethode `schattern()`.

Die `fliege`-Methode von `Ente`-Objekten soll nun so angepasst werden, dass `schnattern()` immer unmittelbar am Anfang von `fliege` aufgerufen wird.

Wie ist das am besten zu realisieren?

```
public void fliege(Ziel z) {
    this.schnattern();
    // hier den Rumpf der fliege-Methode aus Vogel einfuegen
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
    super.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
    this.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
    Vogel.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
}
```

17. 5. Einfach verkettete Listen

`x` sei Knoten einer einfach verketten Liste (genauer: eine Referenz darauf). Wofür sorgt das folgende Codefragment?

```
t.next = x.next;
x.next = t;
```

Knoten `t` wird direkt hinter Knoten `x` eingefügt.

18. 5. Eine einfache Klasse

Programmieren Sie eine einfache Klasse `Buch` mit der Angaben zu Büchern abgespeichert werden können.
Jedes Buch zeichnet sich durch seinen Titel aus, und durch seine Seitenzahl.
Die folgenden Operationen sollen möglich sein:

- Erzeugen einer `Buch`-Instanz unter Angabe des Titels
- Erzeugen einer `Buch`-Instanz unter Angabe des Titels und der Seitenzahl
- Setzen und Abfragen der Seitenzahl jeder einzelnen Instanz.
- Die Anzahl erzeugter Instanzen und die Summe ihrer Seitenzahlen soll durch Methodenaufrufe ermittelt werden können.

VERWENDEN SIE DABEI BITTE DIESE BEZEICHNER:
`titel`, `seiten`, `anzahl`, `gesamt`

Außerdem soll die Klasse für einen ersten Test mit einer `main`-Methode ausgestattet sein, die folgende Aktionen auslöst:

- ein `Buch`-Objekt erzeugen nur unter Angabe des Titels `Lieblingsbuch` und die Referenz der Variablen `b` zuweisen
- anschließend die Seitenzahl dieses Buchobjekts auf `500` setzen
- schließlich mittels `b` die Angaben zu dem erzeugten Objekt ausgeben:

  `Lieblingsbuch, 500 S.`